

LAB 6: Assignment: Implementation of VAE-GAN & WGAN
from
fundamentals with hints
Course Name and instructor: GAI & LLMs , Prof G C Nandi
Date of announcement: 16-02-2026
Full Marks-100

Objective: To implement, train, and compare three generative models of increasing complexity on the MNIST dataset: a standard GAN, a VAE-GAN, and a VAE-WGAN. Students will analyze the strengths and weaknesses of each approach in terms of training stability, sample quality, and latent space properties.

To get an idea about the clumsiness of training a GAN please watch this video(Recommended):

<https://www.youtube.com/watch?v=RzcXri2eJrI>

Dataset: MNIST (handwritten digits, 28x28 grayscale images)

Core Concepts Covered:

- Generative Adversarial Networks (GANs)
- Vanishing Gradients & Mode Collapse
- Variational Autoencoders (VAEs)
- VAE-GAN hybrid architecture
- Wasserstein GAN (WGAN) & Earth Mover's Distance
- Lipschitz Constraint & Weight Clipping
- Latent Space Interpolation

Part 1: Implementation (60% of grade)

Implement the three models below using PyTorch or TensorFlow/Keras. Use the same basic architecture for the Generator/Decoder and Discriminator across all models where possible, to ensure a fair comparison.

General Instructions:

- **Dataset:** Use `tensorflow.keras.datasets.mnist` or `torchvision.datasets.MNIST`. Normalize images to the range `[-1, 1]` (this is standard for GANs as it matches the `tanh` output of the generator).
- **Latent Dimension (`z_dim`):** Set to 100.
- **Batch Size:** 64 or 128.
- **Epochs:** Train for at least 50 epochs for GAN/VAE-GAN, and 100 for VAE-WGAN (as WGANs often need more iterations to converge).
- **Logging:** Save generated images from a fixed random noise vector every 2-5 epochs to visually track progress.

Model 1: Standard GAN

- **Generator (G):**
 - Input: Latent vector `z` (dim=100).
 - Architecture: Dense -> Reshape -> Conv2DTranspose (or UpSampling2D + Conv2D) layers to output a 28x28x1 image.
 - Activation: `ReLU` in hidden layers, `tanh` for the output layer.
- **Discriminator (D):**
 - Input: Image (28x28x1).
 - Architecture: Conv2D layers with LeakyReLU and Dropout, ending with a Dense layer.
 - Activation: `LeakyReLU` in hidden layers, `sigmoid` for the final output (probability of real image).
- **Loss:** Binary Cross-Entropy.

- **Optimizer:** Adam (lr=0.0002, beta_1=0.5) for both G and D.
- **Training Loop:** Standard GAN training. Train D on real and fake batches, then train G to fool D.

Model 2: VAE-GAN

This model combines a VAE with a GAN. The Generator is replaced by a VAE Decoder, and the Discriminator's role is expanded.

- **Components:**
 - **Encoder (E):** Input Image x -> Output: Mean (μ) and Log Variance ($\log_var$) of the latent distribution (dim=100).
 - Architecture: Conv2D layers -> Flatten -> Dense layers for μ and $\log_var$.
 - **Decoder/Generator (G):** Same architecture as the Generator from Model 1. It takes a latent vector z and outputs an image.
 - **Discriminator (D):** Same architecture as Model 1, **but** it is also used as a feature extractor. We will take an intermediate layer's output to compute the perceptual loss.
- **How it works:**
 1. **VAE Path:** An image x is passed through E to get μ and $\log_var$. We sample $z \sim N(\mu, \log_var)$ using the reparameterization trick. G reconstructs the image x_recon from z .
 2. **GAN Path:** A random $z_prior \sim N(0, 1)$ is passed through G to generate a fake image x_fake .
- **Losses (Three Components):**
 1. **VAE Loss (KL Divergence):** $L_{KL} = -0.5 * \sum(1 + \log_var - \mu^2 - \exp(\log_var))$. This encourages the latent space to be close to a standard normal distribution.

2. **Reconstruction Loss (Perceptual Loss):** Instead of pixel-wise L1/L2 loss, we use a feature matching loss from the Discriminator.

- Let $D_{\text{feat}}(x)$ be the output of an intermediate layer of the Discriminator.
- $L_{\text{recon}} = || D_{\text{feat}}(x) - D_{\text{feat}}(x_{\text{recon}}) ||_2$. This forces the reconstructed image to have similar high-level features to the original.

3. **Adversarial Loss (for G and D):** Same as the standard GAN, applied to both x_{recon} and x_{fake} .

- $D_{\text{loss}} = -E[\log(D(x))] - E[\log(1 - D(x_{\text{recon}}))] - E[\log(1 - D(x_{\text{fake}}))]$
- $G_{\text{loss}} (\text{adversarial}) = E[\log(1 - D(x_{\text{recon}}))] + E[\log(1 - D(x_{\text{fake}}))]$ (or use the non-saturating version).

- **Total Losses:**

1. **Encoder Loss:** $L_E = L_{\text{KL}} + L_{\text{recon}}$
2. **Generator Loss:** $L_G = L_{\text{recon}} + L_{G_{\text{adversarial}}}$ (or just $L_{G_{\text{adversarial}}}$, depending on implementation).
3. **Discriminator Loss:** L_D (adversarial as above).

- **Optimizer:** Adam for all components.

Model 3: VAE-WGAN

This model replaces the standard GAN discriminator/critic in the VAE-GAN with a Wasserstein Critic.

- **Components:** Same as VAE-GAN (Encoder E, Decoder/Generator G).
- **Critic (C):** Same architecture as Discriminator D, but with **two crucial changes**:
 1. **No Sigmoid:** The output layer is a linear unit (no activation). It outputs a score (the "criticism"), not a probability.

2. **Loss Function:** It uses the Wasserstein loss.

- **Losses:**

1. **VAE Loss (KL Divergence):** Same as VAE-GAN: $L_{KL} = -0.5 * \sum(1 + \log_var - \mu^2 - \exp(\log_var))$.

2. **Reconstruction Loss (Perceptual):** Same as VAE-GAN, using features from an intermediate layer of the **Critic**. $L_{recon} = ||C_feat(x) - C_feat(x_recon)||_2$.

3. **Wasserstein Loss (for Critic and Generator):**

- Critic Loss = $E[C(x_fake)] - E[C(x_real)] + E[C(x_recon)] - E[C(x_real)]$ (A common simplification is to treat x_recon as part of the "fake" set).
- A more standard formulation:
 - $C_loss = -E[C(x_real)] + E[C(x_fake)] + E[C(x_recon)]$ (but note the sign convention. The goal is for the critic to score real images higher).
- **Generator Loss (Wasserstein):** $G_loss_W = -E[C(x_fake)] - E[C(x_recon)]$ (Generator wants to maximize the critic's score on its generated images).

- **Key Constraint for WGAN:** The Critic must be 1-Lipschitz. Enforce this by **Weight Clipping** (e.g., clamp weights to a small range like $[-0.01, 0.01]$ after each Critic update). *Note: In a full course, you could introduce Gradient Penalty (WGAN-GP) as an advanced improvement.*
- **Training Loop:** Train the Critic ($n_critic = 5$ times) for every one Generator update. Use RMSprop or Adam with a low learning rate (e.g., 0.00005).

Please note that here all hyperparameters mentioned above plays an important role in GAN training

Part 2: Analysis & Report (40% of grade)

Write a comprehensive report (max 10 pages) that includes the following sections:

1. Introduction (10%)

Briefly describe the three models and the motivation behind their development. What problem does each subsequent model aim to solve?

2. Experimental Setup (10%)

Describe the architectures, hyperparameters, and training details for all three models. Include a table summarizing the key differences (e.g., Loss function, Optimizer, Constraint).

3. Results and Comparison (50%)

- **A. Training Stability (15%):**

- Plot the generator and discriminator/critic losses over time for all three models. What do you observe? Which model shows the most stable training? Which shows the most erratic? Discuss the "vanishing gradient" problem in the context of your GAN loss plot.

- **B. Quality of Generated Samples (20%):**

- Show a grid of generated images (e.g., 8x8 grid of random samples) from each model at epoch 1, 10, and 50 (and 100 for WGAN). Comment on the visual quality, diversity, and presence of artifacts.
- **Mode Collapse:** Does any model exhibit signs of mode collapse (e.g., generating only a few types of digits)? Provide evidence.

- **C. Latent Space Interpolation (15%):**

- For the VAE-GAN and VAE-WGAN (which have an encoder), pick two real images from the test set, encode them to get z_1 and z_2 .

Perform linear interpolation in the latent space: $z = (1 - \alpha) * z_1 + \alpha * z_2$ for α from 0 to 1. Decode these interpolated vectors and show the resulting images in a row.

- Perform the same interpolation using two random z vectors for the standard GAN.
- Compare the smoothness and coherence of the interpolations. Which model produces the most semantically meaningful transition? Why?

4. Conclusion (10%)

Summarize your key findings. Which model would you recommend for generating handwritten digits and why? What are the main trade-offs between training stability, sample quality, and implementation complexity?

5. Code Appendix (10%)

Include well-commented code snippets for the core training loops of each model.

Submission Checklist

- Source code (Jupyter Notebook or Python scripts) for all three models.
- A trained model checkpoint (or link to download) for the best-performing model.
- A final report in PDF format containing all the analysis and figures.

Bonus Challenge (Optional, +10%)

Implement **Gradient Penalty (WGAN-GP)** for the VAE-WGAN instead of weight clipping. Show that it leads to even more stable training and better sample quality. Compare the interpolation results and loss curves with your VAE-WGAN (weight clipping) version.

Appendix

Weight Clipping vs. Gradient Penalty in WGANs

Both techniques enforce the **Lipschitz constraint** required for the Wasserstein distance to be valid, but they do it in fundamentally different ways.

Weight Clipping (Original WGAN)

How it works:

After each critic update, you literally clamp all weights to a small range:

python

```
# After optimizer step
for p in critic.parameters():
    p.data.clamp_(-0.01, 0.01) # Force weights into [-0.01, 0.01]
```

Key Characteristics:

- **Direct approach:** Physically restricts weight magnitudes
- **Simple to implement:** Just one line of code
- **Indirect constraint:** Limiting weights indirectly limits the function's Lipschitz constant
- **Global clipping:** Same clipping range for ALL layers/parameters

Problems with weight clipping:

1. **Capacity underuse:** Forces network to use only a tiny fraction of its representational power
2. **Vanishing/exploding gradients:** Can cause gradients to vanish (weights too small) or explode (clipping creates gradient discontinuities)
3. **Optimization difficulty:** Creates a hard boundary that optimization algorithms struggle with
4. **Tuning nightmare:** The clipping parameter c is extremely sensitive (too small = vanishing gradients, too large = no constraint)

Gradient Penalty (WGAN-GP)

How it works:

Adds a penalty term to the loss that enforces the gradient norm to be close to 1:

python

```
def gradient_penalty(critic, real_images, fake_images):
    batch_size = real_images.size(0)

    # Random interpolation coefficient
    alpha = torch.rand(batch_size, 1, 1, 1)

    # Create interpolated images
    interpolated = alpha * real_images + (1 - alpha) * fake_images
    interpolated.requires_grad_(True)

    # Get critic scores for interpolated images
    critic_interpolated = critic(interpolated)

    # Compute gradients
    gradients = torch.autograd.grad(
        outputs=critic_interpolated,
        inputs=interpolated,
        grad_outputs=torch.ones_like(critic_interpolated),
        create_graph=True,
        retain_graph=True
    )[0]

    # Compute gradient penalty:  $(||grad||_2 - 1)^2$ 
    gradient_norm = gradients.view(batch_size, -1).norm(2, dim=1)
    penalty = torch.mean((gradient_norm - 1) ** 2)

    return penalty

# In training loop:
gp = gradient_penalty(critic, real_imgs, fake_imgs)
critic_loss = -torch.mean(critic_real) + torch.mean(critic_fake) + lambda_gp * gp
```

Key Characteristics:

- **Direct constraint:** Explicitly enforces the 1-Lipschitz condition
- **Soft penalty:** Gradient norms are encouraged (not forced) to be 1
- **Local constraint:** Only enforced on interpolated points, which is theoretically sufficient
- **Differentiable:** The penalty is smooth and works well with gradient descent

Advantages:

1. **Better gradient behavior:** No vanishing/exploding gradients from hard clipping
2. **Full capacity:** Network can use its full representational power
3. **Stable training:** Much more stable convergence
4. **No hyperparameter tuning:** λ (typically 10) works well across many problems

Visual Analogy

Think of it like training a dog to stay within a yard:

Weight Clipping: Build an invisible fence that SHOCKS the dog if it tries to leave. Simple but harsh, and the dog might just sit frozen in the middle.

Gradient Penalty: Gently guide the dog with treats when it approaches the boundary. More nuanced, allows exploration, and results in a happier, more capable dog.

Code Comparison

Here's a side-by-side of the training loops:

python

```
# WGAN with Weight Clipping
for epoch in range(epochs):
    for _ in range(n_critic):
        # Train critic
        z = torch.randn(batch_size, z_dim)
        fake_imgs = generator(z)

        critic_real = critic(real_imgs)
        critic_fake = critic(fake_imgs.detach())

        critic_loss = -(torch.mean(critic_real) - torch.mean(critic_fake))
        critic_loss.backward()
        optimizer_critic.step()
```

```

    # Weight clipping - the key difference!
    for p in critic.parameters():
        p.data.clamp_(-0.01, 0.01)

# Train generator
    z = torch.randn(batch_size, z_dim)
    fake_imgs = generator(z)
    generator_loss = -torch.mean(critic(fake_imgs))
    generator_loss.backward()
    optimizer_generator.step()
python

# WGAN-GP with Gradient Penalty
lambda_gp = 10

for epoch in range(epochs):
    for _ in range(n_critic):
        # Train critic
        z = torch.randn(batch_size, z_dim)
        fake_imgs = generator(z)

        critic_real = critic(real_imgs)
        critic_fake = critic(fake_imgs.detach())

        # Compute gradient penalty
        gp = gradient_penalty(critic, real_imgs, fake_imgs.detach())

        # Wasserstein Loss + gradient penalty
        critic_loss = -(torch.mean(critic_real) - torch.mean(critic_fake)) +
lambda_gp * gp
        critic_loss.backward()
        optimizer_critic.step()

        # NO weight clipping!

# Train generator (same as before)
        z = torch.randn(batch_size, z_dim)
        fake_imgs = generator(z)
        generator_loss = -torch.mean(critic(fake_imgs))
        generator_loss.backward()

```

```
optimizer_generator.step()
```

Empirical Differences

In practice, you'll observe:

- **WGAN with weight clipping:** Samples often look "splotchy" or have artifacts; training loss may suddenly spike
- **WGAN-GP:** Cleaner samples; smoother training curves; better diversity

This is why almost all modern WGAN implementations use gradient penalty (WGAN-GP) instead of weight clipping!